LOOPFRAME AI

V4 · FINAL WEB APP BUILD PROMPT

For Google AI Studio · Build mode

PRODUCT PROMISE
Clip review that remembers your corrections and shows its receipts.

LOOPFRAME helps expert-led creators review a small set of transcript-grounded clip candidates, preserve context, save explicit editorial rules, and verify that future recommendations obey those rules.

One product. Three pages in the primary workflow. A real, testable signature loop.

Read this before building

This V4 specification replaces the earlier V1–V3 briefs. It deliberately removes the LinkedIn Personal Brand OS, PM Learning & Portfolio OS, content calendar, performance analytics, automatic publishing and video rendering from the MVP. Do not reintroduce them.

- Build a real multi-page interactive web application, not a single-page static dashboard or a collection of decorative mockups.
- Use a coherent product with three primary navigation areas: Projects, Memory and Library. Settings and Demo Reset belong in the profile menu.
- Prioritise the complete Creator Edit Intelligence loop over optional features.
- Start transcript-first: pasted transcript and a seeded sample project must work without external media services.
- Every visible primary control must perform a real action. No dead buttons, fake charts, pretend uploads or unsupported integration claims.
- Clearly label sample content and cached fallback responses as DEMO DATA / DEMO FALLBACK.
- Use the actual capabilities of the Google AI Studio Build environment. Inspect the generated runtime before selecting packages or integrations.

V4 product decisions

Decision	V4 specification
Product	LOOPFRAME AI, a focused Creator Intelligence Studio.
Positioning	Clip review that remembers your corrections and shows its receipts.
Primary user	Expert-led long-form creators: educators, founder-led podcast hosts and subject-matter experts. This segment is a hypothesis to validate, not proven market demand.
Core pain	Creators spend time reviewing AI-selected clips, correcting exaggerated hooks and checking whether edits preserve the original context.
Signature mechanism	Creator Edit Intelligence: feedback → confirmed rule → saved preference ledger → regenerated candidate → deterministic rule test.
P0 input	Seeded sample project and pasted timestamped transcript.
P0 output	3–5 explainable candidates, context review, hook/caption copy, approve/reject/edit and export.
P0 navigation	Projects, Memory, Library.
Later only	Video/audio upload, transcription, clip rendering, more platforms, publishing and performance analytics.
Explicit non-goals	No PM course, LinkedIn personal-brand module, social publishing, virality prediction, autonomous model training or guaranteed factual correctness.

PART A — MASTER BUILD PROMPT

Copy the complete prompt in Part A into Google AI Studio Build mode after uploading this document. The prompt is the authoritative implementation specification.

```
You are a senior product designer, UX architect, frontend engineer, backend engineer and AI systems engineer.
Build LOOPFRAME AI V4 as a complete, polished, multi-page, interactive web application.

DO NOT build a single-page static dashboard. DO NOT return only a plan or mockup. Implement the application,
navigation, routes, persistent state, functional controls, seeded sample data, validations, responsive layouts
and error states.

============================================================
1. PRODUCT DEFINITION
============================================================

Name: LOOPFRAME AI
Positioning: Clip review that remembers your corrections and shows its receipts.

LOOPFRAME is a focused Creator Intelligence Studio for expert-led creators who turn long-form recordings into
short-form content. It helps them:
1. Review a small set of explainable clip candidates derived from a timestamped transcript.
2. Inspect source context before approving a clip.
3. Correct an AI suggestion using natural language.
4. Convert the correction into a specific, editable rule.
5. Confirm and save that rule in a visible preference ledger.
6. Generate or re-rank another candidate using saved rules.
7. Run a deterministic test to verify whether the new output obeys the rule.
8. Approve and export the content.

The product is NOT a generic AI video clipper. Do not claim that clipping, captions, brand kits or calendars are
novel. The proposed differentiator is a user-visible, user-editable preference ledger connected to source-
grounded clip review and post-generation rule verification. Treat differentiation and market demand as
hypotheses until validated with creators.

Primary users for the MVP:
- Educators publishing long-form lessons or discussions.
- Founder-led podcast hosts.
- Subject-matter experts publishing interviews or explainers.

These segments are initial hypotheses. Do not state that they are validated.

Core user promise:
“Spend less time repeating the same corrections. See what the AI selected, where it came from, which preference
it followed, and whether the result passed the rule test.”

============================================================
2. NON-NEGOTIABLE SCOPE
============================================================

IN SCOPE FOR P0:
- Multi-page interactive web application.
- Three main navigation areas: Projects, Memory, Library.
- Minimal creator onboarding (maximum 3 fields).
- Seeded sample project that works immediately.
- Paste/import a timestamped transcript as the primary input.
- Generate 3–5 clip candidates from transcript segment IDs.
```

```
- Explainable candidate cards with source timestamps and exact source excerpts.
- Context review with deterministic checks and clearly labelled AI advisory checks.
- Creator feedback and a closed rule taxonomy.
- User confirmation before saving a preference.
- Visible preference ledger with edit, disable, delete and undo/rollback.
- Regenerate/re-rank candidates using confirmed rules.
- Deterministic post-generation rule tests.
- Hook and caption generation for two initial destinations: Instagram Reels and YouTube Shorts.
- Human approval and a functional export of copy and timecodes.
- Demo fallback that is deterministic and labelled.

OUT OF SCOPE FOR P0:
- PM Learning & Portfolio OS.
- LinkedIn Personal Brand OS as a separate module.
- Content calendar.
- Performance analytics and performance memory.
- Automatic publishing.
- Video rendering and downloadable video files.
- Live social-account connections.
- Virality scores or guaranteed reach predictions.
- Fine-tuning or autonomous model training.
- Multi-user team collaboration.
- Complex billing or subscription features.

Do not add out-of-scope features to the navigation.

============================================================
3. MULTI-PAGE INFORMATION ARCHITECTURE
============================================================

Build real routes/pages with a persistent application shell.

PRIMARY NAVIGATION:
1. Projects
2. Memory
3. Library

PROFILE MENU:
- Creator profile
- Settings
- Demo mode / Reset sample data
- Help / About

Required pages:

PAGE 1 — Welcome / lightweight onboarding
- A concise explanation of the product.
- Primary CTA: “Open sample project”.
- Secondary CTA: “Create a project”.
- Ask no more than three optional fields: creator/channel name, content category, preferred tone.
- Allow skip.
- Never block access to the sample project.

PAGE 2 — Projects home
- Project list with project name, source type, last updated, candidate count and status.
- “New project” action.
```

- “Open sample project” card.
- Empty state with a clear next action.
- Search/filter only if functional.
- Project cards open a dedicated project workspace route.

PAGE 3 — New project
- Project name.
- Source input choices: “Use sample transcript” and “Paste transcript”.
- Transcript paste area with clear timestamp format guidance.
- Validate transcript structure.
- Create project and navigate to its Studio page.
- Do not show a fake file upload for video in P0.

PAGE 4 — Project Studio (the core workspace)
This is the most important page. Use a professional three-pane desktop layout:
- Left pane: candidate list (3–5 cards).
- Centre pane: transcript with timestamps and highlighted source segments; a simple waveform/timeline visualization may be used only if it reflects actual segment timestamps.
- Right pane: selected candidate details, context checks, feedback, rule confirmation and copy.

On smaller screens, stack panes in a logical sequence with sticky primary actions. Do not squeeze three columns into tiny mobile panels.

Candidate card must show:
- Candidate title.
- Start/end timestamps and duration.
- One-sentence reason for selection.
- Context status.
- Review status.
- Platform copy status if available.

Candidate detail must show:
- Exact source excerpt assembled from transcript segment IDs.
- Surrounding transcript before and after the candidate.
- Why the segment was selected.
- Potential context risk.
- Hook and caption fields.
- Actions: Approve, Edit, Reject, Flag for context review.
- Feedback field: “What should change next time?”

PAGE 5 — Memory
A dedicated preference ledger. This is not a generic settings page.

Each preference/rule row shows:
- Human-readable rule.
- Rule type.
- Scope: global, project or platform.
- Source feedback that created it.
- Created/updated timestamp.
- Status: active, paused or deleted.
- Confirmation status.
- Edit, pause, delete and undo where applicable.

Show a clear explanation:
“LOOPFRAME does not silently retrain itself. It stores confirmed preferences and applies them as explicit rules during future generation.”

```
PAGE 6 — Library
- Approved content only by default.
- List/grid of approved candidate content.
- Search by project/title.
- Filter by platform and status only if functional.
- Each item opens its source project and approved copy.
- Export actions for caption text, source excerpt and timecodes.
- Status values: Needs review, Approved, Rejected.
- No calendar or publishing schedule.
```

```
PAGE 7 — Settings / profile
- Edit the three onboarding fields.
- Demo mode indicator.
- Reset sample data with confirmation.
- Data/privacy explanation that accurately reflects the actual persistence implementation.
- Export user data if implemented.
- Do not claim deletion or encryption guarantees that are not implemented.
```

```
Navigation acceptance:
- Every nav item changes route/page.
- Browser back/forward works.
- Refreshing a nested route does not break the app.
- Current page is visibly highlighted.
- Empty, loading, success and error states exist.
- No nav item is decorative.
```

```
============================================================
4. UI/UX DESIGN SYSTEM — HIGH PRIORITY
============================================================
```

```
Design a distinctive premium SaaS application. It must look like a real product, not a generic AI dashboard
template.
```

```
Visual direction:
- Light-first creator workspace with a calm editorial feel.
- Warm off-white or very light neutral page background.
- White content surfaces with subtle borders.
- Deep ink/navy text.
- Restrained electric-cyan or teal accent for primary actions.
- Violet only for AI-related secondary highlights.
- Green for verified rule-test success; amber for review; red for failure.
- Avoid excessive gradients, neon cyber styling, glassmorphism overload and decorative charts.
- Use a clean modern sans-serif such as Inter.
- Strong typographic hierarchy, generous whitespace, consistent spacing.
- Card radius around 12–16px; subtle shadows only where useful.
- Use consistent outline icons.
- Use skeletons only for actual loading.
- Do not use random stock images as fake video content.
```

```
Application shell:
- Persistent left sidebar on desktop, approximately 240–260px wide.
- Top bar with project breadcrumb, page title/context, profile menu and a contextual primary action.
- Main content area with readable max width and responsive grid.
- Sidebar brand mark and “LOOPFRAME” wordmark.
- Nav labels: Projects, Memory, Library.
```

- Small demo badge when seeded data is active.

Dashboard visual hierarchy:
1. Page title and one clear primary action.
2. Recent projects.
3. A compact “Memory rules active” summary.
4. A helpful next-step panel.
Do not add vanity KPI cards such as total views, viral score or engagement growth.

Project Studio visual hierarchy:
1. Candidate list.
2. Transcript/source evidence.
3. Context check and rule-aware review.
4. Feedback/action panel.
Use a clear selected state and keep Approve/Reject/Save actions easy to find.

Micro-interactions:
- Hover/focus states.
- Selected candidate transition.
- Toast after saved preference, approval, deletion or export.
- Confirmation modal for destructive actions.
- Undo toast after preference deletion where technically supported.
- Keyboard accessible dialogs and controls.
- Clear disabled states with explanation.
- No fake progress bars.

Responsive requirements:
- Desktop: full three-pane Studio.
- Tablet: candidate list collapses or becomes a drawer; transcript and detail remain readable.
- Mobile: single-column workflow with tabs/accordions for Candidates, Transcript and Review.
- No horizontal overflow.
- Touch targets should be comfortable.
- Maintain visible focus and semantic labels.

5. TRANSCRIPT-FIRST PROJECT INPUT

P0 input must work without external APIs.

Support:
A. Seeded sample transcript.
B. User-pasted timestamped transcript.

Use a predictable timestamped format, for example:
[00:00:05] Speaker 1: ...
[00:00:12] Speaker 1: ...

If speaker labels are missing, do not invent them.
If timestamps are malformed or non-monotonic, show the exact lines that need correction.
Parse transcript into immutable segments with:
- segment_id
- start_ms
- end_ms (if supplied or derived safely)
- speaker_label (nullable)
- text

```
- source_order

Preserve original user-provided text. Do not ask an LLM to reproduce or rewrite source quotes.

For P0, no raw video upload is required. Clearly explain that the current project is transcript-first.
Video/audio upload and transcription belong to P1 after the core workflow works.

============================================================
6. CANDIDATE GENERATION — SOURCE-GROUNDED
============================================================

Generate only 3–5 candidates per transcript.

The model must return structured JSON containing:
- candidate_id
- title
- start_segment_id
- end_segment_id
- reason
- suggested_hook
- platform_targets
- risk_flags
- candidate_type

The server must:
1. Validate the JSON schema.
2. Verify start/end segment IDs exist.
3. Verify start comes before end.
4. Build the source excerpt itself from the selected transcript segments.
5. Derive timestamps from the transcript segments.
6. Calculate duration from timestamps.
7. Reject unknown segment IDs.
8. Never display a model-written quote as if it were an exact source quote.

The model must not invent timestamps or source text.
If output fails validation, retry once with a correction instruction. If it fails again, use the labelled
deterministic demo fallback or show a useful error.

Every candidate must show:
- source excerpt built by code
- source segment IDs internally
- timestamp range
- duration
- selection reason
- context status
- uncertainty where appropriate

No virality score. No claims of guaranteed engagement.

============================================================
7. CONTEXT INTEGRITY CHECKS
============================================================

Do not label a candidate “factually safe” or “guaranteed accurate”.

Use status labels:
```

- “No issues found by N checks”
- “Review recommended”
- “Source mismatch”

Deterministic checks:
1. Source segment IDs exist.
2. Segment order is valid.
3. Source excerpt is assembled from original segments.
4. Start/end timestamps are valid and duration is within configured limits.
5. Candidate does not begin/end at an obviously incomplete segment if detectable by simple punctuation/rule checks.
6. Check a configurable window around the clip for context-changing terms such as “but”, “however”, “although”, “unless”, “except”, “not”, “never”, “only”, “because”, “instead”.
7. Check whether numbers, named entities and negations introduced in the generated hook/caption are supported by the selected excerpt or clearly marked for review.

AI-advisory checks:
- Potential pronoun/reference ambiguity.
- Whether the hook sounds more absolute or exaggerated than the source.
- Whether the excerpt may omit an important qualification.
- Whether the caption changes the speaker’s intended meaning.

Clearly label these “AI advisory”, not deterministic validation.

If uncertain, use “Review recommended”.
If source segment IDs are invalid, use “Source mismatch”.
Never use a green PASS badge that implies factual correctness.

Allow the creator to expand the candidate by adding surrounding transcript segments. Re-run checks after changes.

8. SIGNATURE DIFFERENTIATOR — CREATOR EDIT INTELLIGENCE

This is the product. Implement it end to end.

Required example flow:
1. A candidate is generated with an exaggerated hook.
2. Creator rejects it and enters: “Avoid exaggerated hooks. Keep the original educational context.”
3. The AI maps this feedback to a proposed rule from a closed taxonomy.
4. The app shows the proposed rule in plain language.
5. The creator confirms or edits the rule.
6. The confirmed rule is saved to the preference ledger with source feedback, scope and timestamp.
7. A new candidate/hook is generated with the active rule included in the model context.
8. Deterministic code tests the new output against the rule.
9. The UI shows the rule, the output, and the actual test result.
10. If the test fails, do not claim the preference was applied. Show failure and offer revise/retry.

Closed rule taxonomy for P0:
- AVOID_EXAGGERATED_HOOKS
- AVOID_CLICKBAIT
- PRESERVE_QUALIFICATIONS
- PRESERVE_EDUCATIONAL_CONTEXT
- AVOID_UNSUPPORTED_CLAIMS
- PREFER_CONCISE_HOOKS

- PREFER_CONVERSATIONAL_TONE
- AVOID_JARGON
- INCLUDE_CLEAR_TOPIC
- CUSTOM_REVIEW_REQUIRED

Each rule has:
- rule_id
- rule_type
- human_readable_description
- scope (global/project/platform)
- scope_reference
- source_feedback_id
- created_at
- updated_at
- status (active/paused/deleted)
- confirmed_by_user
- rule_parameters (structured fields)

Do not treat arbitrary free text as an enforceable rule. Map it to the closest supported rule and show the mapping for user confirmation. If no reliable mapping exists, save it as a note requiring manual review, not as an active enforcement rule.

Examples of deterministic tests:
- AVOID_EXAGGERATED_HOOKS: check a configurable lexicon of superlatives/absolute phrases; show the matched term. This is a heuristic, not a complete language understanding system.
- PREFER_CONCISE_HOOKS: enforce a user-configured maximum word count.
- AVOID_JARGON: test against a user-maintained jargon list.
- INCLUDE_CLEAR_TOPIC: require a selected topic keyword or explicit topic phrase.
- PRESERVE_QUALIFICATIONS: check whether required qualification terms from source context are retained or explicitly flagged.

For rules that cannot be reliably tested with deterministic logic, show “AI advisory only” and do not claim a deterministic pass.

Preference controls:
- Confirm.
- Edit.
- Pause.
- Delete.
- Undo recent deletion if supported.
- Change scope.
- View source feedback.
- View when it was last applied and its last test result.

Preference scope:
- Global: applies to all projects.
- Project: applies only to one project.
- Platform: applies only to one destination.

Default to project scope unless the creator explicitly chooses otherwise.

Explicit rules take precedence over inferred suggestions.
Do not silently create or activate a rule.
Do not claim model training or fine-tuning. The system stores confirmed rules and injects them into generation context.

==========

9. RULE VERIFICATION AND EVALUATION

```
============================================================

The product must prove rule compliance, not merely narrate it.

After generation:
- Display the exact active rules used.
- Run each applicable deterministic test against the new hook/caption.
- Show Pass, Fail or Advisory.
- Show the matched phrase or reason.
- If failed, provide a revise action.
- Never hide a failed test.

Add an evaluation workspace or project-level test panel:
- Seed 20–30 correction test cases.
- Include examples for each P0 rule type.
- Include negative cases where a rule should not trigger.
- Record whether feedback was mapped correctly.
- Record whether the user confirmed the proposed rule.
- Record whether the regenerated output passed the rule test.
- Record false positives and false negatives.
- Record whether source meaning/qualifications were preserved.

Do not invent accuracy percentages. Show actual counts from the test cases. Label the dataset “small prototype evaluation”.
For the live demo, include a resettable seeded scenario and clearly distinguish live model output from cached deterministic fallback.

============================================================
10. FEEDBACK AND REVIEW WORKFLOW
============================================================

Candidate actions:
- Approve.
- Edit.
- Reject.
- Flag for context review.
- Expand source context.

When rejecting/editing:
- Provide concise reason chips: exaggerated hook, missing context, wrong tone, irrelevant, too long, jargon, other.
- Provide a free-text feedback field.
- Do not force a long explanation.
- Store feedback event with timestamp and candidate ID.
- If feedback maps to a supported rule, propose it for confirmation.
- If not, store as unstructured feedback note.

Approved items move to Library.
Rejected items remain visible in project history but are excluded from Library by default.
Edited items preserve revision history.

============================================================
11. PLATFORM COPY — TWO DESTINATIONS ONLY
============================================================

P0 destinations:
```

```
- Instagram Reels.
- YouTube Shorts.

For each candidate generate:
- Hook.
- Caption.
- Optional CTA.
- Optional hashtags only when useful.

Platform differences:
- Instagram: concise, conversational, creator-native.
- YouTube Shorts: clear topic framing and searchable title/caption.

Do not claim knowledge of proprietary ranking algorithms.
Do not promise reach, virality or engagement.
Allow manual editing.
Run applicable creator rules against both hook and caption.
Show source excerpt beside the copy so the creator can inspect whether meaning is preserved.

LinkedIn is not a separate module in V4. Do not build LinkedIn personal branding, professional case studies or a
LinkedIn content workspace.

============================================================
12. MULTI-PAGE DATA MODEL AND PERSISTENCE
============================================================

First inspect the generated environment and choose one coherent persistence approach.

Preferred simple path:
- Firebase Authentication + Firestore if Google AI Studio's current environment supports the integration
cleanly.
- Otherwise use a supported persistent database adapter.
- For a local-only prototype, use a clearly labelled local demo store and isolate persistence behind an adapter.

Do not specify a relational schema if implementing Firestore. Use collections/documents consistently.

Suggested Firestore collections:
- users/{userId}
- users/{userId}/creatorProfile
- users/{userId}/projects/{projectId}
- users/{userId}/projects/{projectId}/transcriptSegments/{segmentId}
- users/{userId}/projects/{projectId}/candidates/{candidateId}
- users/{userId}/projects/{projectId}/feedbackEvents/{feedbackId}
- users/{userId}/preferences/{preferenceId}
- users/{userId}/libraryItems/{itemId}
- users/{userId}/auditEvents/{eventId}

Every user-owned document must include ownership and timestamps as appropriate.
Implement per-user security rules. Users must not read or write another user's records.
Do not expose server secrets or API keys in frontend code.
If authentication cannot be completed in the build environment, clearly label the app as a single-user demo and
do not claim multi-user security.

Persist:
- projects
- transcript segments
```

```
- candidates
- feedback
- confirmed preferences
- approvals/rejections
- library items
- audit events

Handle loading, save failure, retry and stale state.

============================================================
13. GOOGLE AI STUDIO IMPLEMENTATION GUIDANCE
============================================================

Inspect the actual runtime before implementing.

Use the framework and server runtime supported by Google AI Studio Build mode. A React frontend and Node server runtime may be available, but verify rather than assume.

Use server-side model calls and environment-managed secrets.
Use structured JSON output and runtime schema validation.
Do not expose API keys to the browser.
Do not invent package names or unsupported APIs.
Do not assume FFmpeg, large-file uploads, long-running jobs or object storage are available.
Do not build video rendering in P0.

Server functions/routes should be modular:
- createProject
- parseTranscript
- generateCandidates
- validateCandidate
- parseCreatorFeedback
- confirmPreference
- applyPreferences
- testGeneratedOutput
- generatePlatformCopy
- approveCandidate
- exportContent

If model calls fail:
- show a clear error;
- allow retry;
- use seeded fallback only when clearly labelled;
- never silently present cached content as live AI output.

Add request limits, timeouts, quota handling and safe error messages.
Keep data ownership checks on the server as well as in database rules.

============================================================
14. SEEDED DEMO PROJECT
============================================================

Create one original sample podcast transcript about an educator explaining how to learn complex topics. Do not use copyrighted podcast transcripts or real-person quotations.

Include:
- 12–18 timestamped transcript segments.
```

- 3 initial clip candidates.
- One candidate with a missing qualification/context warning.
- One candidate with an exaggerated hook.
- One candidate that is acceptable.
- One creator feedback event rejecting exaggerated hooks.
- One proposed rule awaiting confirmation.
- A confirmed rule scenario that can be activated by a demo action.
- A regenerated candidate that passes the deterministic rule test.
- At least one failed rule test example so the UI demonstrates honest failure handling.

All sample records must display DEMO DATA.
Any cached response must display DEMO FALLBACK.
Provide “Reset demo” with a confirmation dialog.
The demo must work without an API key or uploaded media.

15. VISUAL CONTENT AND INTERACTION DETAILS

Projects page:
- Modern project cards with title, last edited, candidate count and status.
- A prominent “New project” button.
- A visually distinct sample project card.
- Useful empty state.

Studio:
- Candidate cards with selection state and status.
- Transcript segment highlighting tied to selected candidate.
- Surrounding context visible without extra navigation.
- Context check panel with deterministic vs AI-advisory labels.
- Feedback input and concise reason chips.
- Proposed preference confirmation panel.
- Rule test result visible after regeneration.
- Approve and export actions.

Memory:
- Search/filter preferences only if functional.
- Active/paused/deleted tabs or filters.
- Scope chips.
- Source feedback preview.
- Last applied and last test result.
- Edit/pause/delete/undo interactions.

Library:
- Approved content cards.
- Preview hook/caption.
- Source project and timestamp.
- Copy text, export text and export timecodes.
- No schedule or performance charts.

Toasts:
- Preference saved.
- Candidate approved.
- Export ready.
- Rule test failed.
- Changes saved.

Use accessible live regions where practical.

==

16. ACCEPTANCE CRITERIA — MUST ALL BE TESTED

==

1. The application is a multi-page website with Projects, Memory and Library routes, not a static one-page dashboard.
2. Navigation works and browser refresh/back/forward do not break nested routes.
3. A first-time user can open the sample project without onboarding friction.
4. A user can create a project using a pasted timestamped transcript.
5. Transcript parsing validates timestamps and preserves source text.
6. The app generates or loads 3–5 candidates with valid segment IDs.
7. Source excerpts and timestamps are assembled by code, not invented by the model.
8. The user can inspect surrounding context and see deterministic checks separately from AI advisory checks.
9. The user can reject/edit a candidate and submit feedback.
10. The app proposes a rule from the closed taxonomy and waits for user confirmation.
11. A confirmed rule appears in Memory with source, scope and timestamp.
12. The user can edit, pause and delete a preference.
13. A subsequent generation uses active applicable rules.
14. The app runs deterministic post-generation tests and shows pass/fail/advisory honestly.
15. A failed test is visible and does not get described as successful.
16. Approved candidates appear in Library.
17. The user can edit and export hook/caption/timecodes.
18. Demo data and fallback outputs are visibly labelled.
19. No video rendering, auto-publishing, virality prediction or PM/LinkedIn modules are falsely shown as implemented.
20. Desktop, tablet and mobile layouts are usable and accessible.

==

17. BUILD ORDER

==

Build in this order. Keep the app runnable after each milestone.

MILESTONE 1 — Design system and multi-page shell
- Build responsive sidebar, top bar, Projects, Memory and Library routes.
- Build welcome, onboarding, profile menu and empty states.
- Establish visual system before adding dense content.

MILESTONE 2 — Transcript-first Projects
- Build create project flow.
- Add seeded sample transcript.
- Parse timestamped transcript.
- Build project Studio and transcript display.

MILESTONE 3 — Candidate generation and context checks
- Generate or load 3–5 candidates.
- Use segment IDs.
- Assemble source quotes in code.
- Add deterministic checks and advisory labels.

MILESTONE 4 — Creator Edit Intelligence
- Feedback input.
- Closed rule taxonomy.
- Rule proposal and user confirmation.

```
- Preference ledger.
- Apply rules to generation.
- Deterministic post-generation rule tests.
- Undo/pause/delete.

MILESTONE 5 — Library and export
- Approval flow.
- Approved content library.
- Copy/export text and timecodes.

MILESTONE 6 — Persistence, security and QA
- Add supported persistence/auth.
- Add per-user rules if auth is implemented.
- Add request error handling and demo fallback.
- Test every route, action and responsive breakpoint.

Do not start P1 features until the full P0 workflow passes.

============================================================
18. FINAL BUILD AND QA INSTRUCTIONS
============================================================

Start building the application now.

Before declaring completion:
- Run the application.
- Fix compilation and runtime errors.
- Test every route and primary control.
- Test the complete signature loop: candidate → correction → proposed rule → confirmation → saved memory →
regenerated candidate → deterministic rule test.
- Test a failed rule case.
- Test project creation from pasted transcript.
- Test export.
- Test persistence after refresh where supported.
- Test desktop, tablet and mobile layouts.
- Check keyboard focus and accessible labels.
- Verify no unsupported features or fake metrics are shown.
- Verify all sample content and fallback outputs are labelled.

Provide a concise README with:
- how to run the app;
- architecture and persistence choice;
- environment variables;
- demo instructions;
- what is genuinely implemented;
- what is simulated;
- known limitations;
- P1 roadmap.

If any feature cannot be implemented, do not fake it. Complete the core loop first and report the limitation
honestly.

Build LOOPFRAME AI V4 now.
```

PART B — FOLLOW-UP PROMPTS FOR GOOGLE AI STUDIO

1. Full multi-page navigation audit

Audit the application as a real multi-page website. Test Projects, Memory and Library routes, nested project routes, refresh, browser back/forward, mobile navigation and all primary actions. Fix broken navigation, dead buttons, inconsistent page shells and any route that renders a static placeholder. Do not add new features.

2. Creator Edit Intelligence proof test

Run the complete signature flow. Start with an exaggerated hook candidate, enter a creator correction, map it to a closed rule, show the proposed rule, confirm it, save it in Memory, regenerate a candidate, and run the deterministic test. Show the exact rule and matched phrase. Demonstrate one successful and one failed test. Never claim compliance when the test fails.

3. UI/UX red-team pass

Review the current app as a senior SaaS product designer. Improve hierarchy, spacing, typography, responsive layouts, empty states, selected states, feedback forms, dialogs and accessibility. Keep the three-page navigation and existing product scope. Do not turn the app into a single-page dashboard. Remove decorative charts and nonfunctional controls.

4. Data and security audit

Inspect persistence and ownership. If Firebase Auth/Firestore is implemented, verify per-user security rules and server-side ownership checks. If not, label the app as a single-user demo. Test refresh persistence, save failures, retries, reset demo and data isolation. Do not claim security guarantees that are not implemented.

5. Final release audit

Run a full acceptance test against all 20 criteria in the V4 brief. Fix all P0 defects. Report each criterion as PASS, FAIL or NOT IMPLEMENTED with evidence. List simulated features and known limitations. Do not add P1 features before P0 is complete.

PART C — THREE-MINUTE DEMO SCRIPT

0:00–0:20 | Positioning — “AI can find clips. LOOPFRAME focuses on what happens after the creator says, ‘That is not how I want my content represented.’”

0:20–0:55 | Projects and source evidence — Open the sample project. Show three candidates, their exact transcript excerpts, timestamps and reasons. Open the candidate with a context warning and inspect the surrounding transcript.

0:55–1:30 | Creator correction — Reject the exaggerated hook and enter: “Avoid exaggerated hooks. Keep the original educational context.” Show the proposed rule.

1:30–2:05 | Memory — Confirm the rule. Open Memory and show its scope, source feedback, timestamp and active status.

2:05–2:35 | Verification — Regenerate the candidate. Show the actual deterministic rule test, including the matched phrase or pass result. If it fails, show the failure honestly.

2:35–3:00 | Library and close — Approve the candidate, open Library, and export the hook/caption/timecodes. Close: “LOOPFRAME remembers your corrections and shows whether it followed them.”

PART D — P1/P2 ROADMAP (NOT FOR INITIAL BUILD)

Phase	Possible additions	Gate before starting
P1	Audio/video upload and transcription; preference scope refinement; second platform; held-out 20–30 case evaluation.	All P0 acceptance criteria pass and the core loop is stable.
P2	Clip rendering; publishing integrations; performance data import; calendar; separate LinkedIn product exploration.	Real creator feedback demonstrates demand and technical feasibility.

PART E — FINAL HONESTY CHECK

- Do not claim the system has trained or fine-tuned a model. It stores confirmed rules and injects them into generation context.
- Do not claim a rule was applied unless the corresponding test ran and passed.
- Do not call an AI advisory check deterministic.
- Do not claim source content is factually safe because no issues were found.
- Do not claim video upload, transcription or rendering unless those capabilities genuinely work.
- Do not show virality scores, engagement uplift or performance analytics in P0.
- Do not claim customer demand or validation without actual creator interviews and evidence.
- Do not claim multi-user security unless authentication, ownership checks and database rules are implemented and tested.
- Do not reintroduce PM Learning OS or LinkedIn Personal Brand OS into this V4 MVP.